

Annex VII — Handshake Core, Profiles, Capability Manifests, Grants, and pBEAP™

Optirando™ Canon — Normative Annex VII. Version 1.1 (see Section 17). Depends on: the Optirando™ Canon; Annex II; Annex III (PoAC™); Annex IV (Field Operation Modes, DeepFix); Annex V (Media Reconstruction Model); Annex VI (WR Preparation Stages); Annex IX (Initiation, Public Scalability, and Transport Privacy)

0. Purpose and Position in the Canon

This annex specifies (a) the permanent wire-format core of the BEAP™ handshake, (b) the handshake profiles governing all relationship types, (c) the capability layer — how automations declare, verify, distribute, and anchor LoRA adapters, templates, and capability scopes into the WR Graph™, (d) the grant model governing every right a publisher can hold toward a consumer, and (e) pBEAP™, the asymmetric public-offering profile, together with the Private Distribution Path for non-public automations.

The core philosophy of the Canon applies without exception and is, in this annex, extended to relationships themselves: the system prepares, the human authorizes, the system records cryptographic proof. No state of the system exists that a human did not bring about.

0.1 Normative Language

MUST, MUST NOT, SHALL, SHOULD, SHOULD NOT, MAY per RFC 2119. Requirements are numbered VII-⟨section⟩.⟨n⟩.

1. Terminology

Provider / Publisher. The party offering an automation. In pBEAP™ contexts the term Publisher is used; it is always an organization, never a private person.

Consumer. The party receiving an offering on their own WR Desk™ installation. May be a private person or an organization.

Artifact. A distributable unit: LoRA adapter (safetensors), session/execution template, or bridge adapter.

Adapter Manifest. The signed, structured, machine-readable declaration of an adapter's identity, geometry, base-model binding, scope, and bridge compatibility (Section 6).

Manifest Root. A Merkle root computed over all of a provider's published artifact commitments. The root — not individual leaves — is the object exchanged in the handshake and, in the future, anchored on public ledgers (Sections 5, 7).

Capability Diff. The mandatory UI surface that renders, on every capability ingress path, the requirements of an offering against the local system state (Section 14).

Ingress Path. Any channel by which a capability claim reaches a consumer: the BEAP™ handshake, a WR Code® scan, a pBEAP™ message, or a BEAP™ message received on any other channel.

Initiation Class. The registry-defined class of the invitation, claim, and consent mechanics by which a handshake request comes into being (Annex IX: `public_bearer`, `targeted_bound`). Orthogonal to the Handshake Profile (Section 4.6).

Bridge Adapter. An adapter whose declared function is transforming or substituting compatibility between base models (Section 6.3).

Handshake Profile. The single, immutable, signed discriminator of a handshake that normatively fixes its mandatory fields, permitted ingress paths, role symmetry, required signature count, and permitted grants (Section 4).

Account Class. The entitlement tier of an account — Free, Pro, Publisher, Enterprise — gating which profiles the account may initiate in the offering/organizational role (Section 4.5). Consumer-role capabilities are class-independent.

Extension. A namespaced, versioned, criticality-flagged entry in the handshake's or a grant's extensions container (Section 3.2).

Grant. A right held by a publisher toward a consumer, created exclusively by explicit consumer consent: a delivery right, a standing action scope, or both. Grants never include execution authority (Section 10).

Standing Action Scope. A grant pinning one exact template version and its constraints, permitting the publisher to deliver payloads that the consumer's client renders as prepared actions (Section 10.3).

Prepared Action. A fully verified, staged, effect-transparent action awaiting a human consent tap. Execution of a prepared action produces a PoAE™ record.

Session. An organizational and display construct in the WR Graph™ (e.g., a hotel stay, a machine deployment) with optional hierarchy. Sessions organize; they do not govern grant lifetimes (Section 11).

Red WR Code®. A WR Code® whose payload is sealed to organizational or personal keys, rendered with the red internal color coding (Section 4.4).

Relationship. The totality of handshake, grants, and graph region between one attested publisher identity and one consumer.

2. Architectural Principles

VII-2.1 — Root in the handshake, leaves in messages — with the Variant-A exception. The handshake SHALL establish provider identity, registry namespace, supported base models, and the signed Manifest Root. Individual artifact leaves are carried by the ingress message that requires them — except under Declaration Variant A (Section 5.2), where the scanned code itself supplies offering-level granularity and the handshake request MAY carry that one offering's leaves inline, in addition to the root. The catalog-blob prohibition stands: no ingress form ever carries a provider's artifact catalog.

VII-2.2 — No string is authoritative. Compatibility, scope, and identity decisions are made exclusively from verified manifest fields, never from artifact names, filenames, or display labels

(Section 6). Canonical names are projections of manifest content and are normatively non-authoritative.

VII-2.3 — Verification is a chain, not a lookup (Section 7).

VII-2.4 — No silent state transition — extended to relationships. Rights come into existence only by explicit consent, change only by explicit consent (including self-consented limits), end only by revocation or a consented limit, and every execution is a separately consented act. Automatic teardown of relationships does not exist (VII-10.7.1).

VII-2.5 — Trust-root separation. Verified safetensors adapters MAY load on the host after the full Section 7 chain. Tools are ALWAYS sandbox-routed regardless of verification outcome. Original media never enters the Host trust root (Annex V).

VII-2.6 — No execution authority is ever grantable. No grant, scope, profile, or extension confers the right to execute anything on the consumer's system. Grants confer delivery and preparation rights only. Execution is always consent-gated at the instant of the human's tap (VII-10.1).

VII-2.7 — Receiver-enforced admission. All delivery rights are enforced by the consumer's client at ingress. Delivery exists only under a granted right; absent a right, publisher transmissions are discarded before any user visibility. There is no protocol state in which any party sends unrequested content that reaches a user. (Informative: this inverts the email model, where the sender controls delivery and the recipient's address is an open port. Here the recipient holds both gates: delivery is grant-gated, execution is consent-gated.)

VII-2.8 — Profiles, not flags. Relationship semantics are fixed by exactly one profile per handshake, never by field combinations (Section 4).

3. Handshake Core (Wire Format)

The handshake core defined in this section is intended to be permanently stable. All future evolution occurs through the profile registry (Section 4.1), the extensions container (Section 3.2), the declarations container (VII-3.1), registered ingress identifiers (VII-3.1), and grant vocabularies (Section 10.4) — never through core schema changes.

3.1 Signed Core Structure

VII-3.1 — The handshake record consists of a signed core and a signature list. The signed core contains, at minimum:

Field	Content
profile	Handshake profile identifier + profile_version (Section 4). Immutable for the life of the relationship.
initiator_id / responder_id	SSO-bound identities (Section 3.3)
ingress_path	How this handshake was initiated: a registry-backed identifier from the Optirando™ ingress registry (namespace optirando.ingress.*) — recorded for PoAC™; MUST NOT alter handshake semantics (VII-4.6). Unknown identifiers are preserved and ignored (the VII-6.1.5 rule); the identifier never enters any semantic decision. Initial registry entries: wr_code_public, wr_code_red, beap_invitation, relay_code_claim, optirando_code_entry. Reserved: wr_ad.

Field	Content
declarations	Generic declarations container: an ordered list of namespaced, versioned declaration payloads — [{ns, version, payload}] — following the identical preservation and criticality discipline as the extensions container (VII-3.4, VII-3.5). Declaration namespaces are registered in the declaration registry (companion to the profile and extension registries). The capability declaration of Section 5 is carried as the registered namespace optirando.decl.capability, version 1, whose payload is the Variant A or Variant B form of Section 5 unchanged.
extensions	Extensions container per Section 3.2
created_at, nonce	Freshness and replay protection

VII-3.2 — Signature list. The core carries an ordered list of 1..n detached signatures. Each profile declares normatively how many signatures, from which roles, are required for the handshake to be established. This is a wire-format core decision made now precisely because signature cardinality cannot be retrofitted as an extension.

- **pbeap_publisher:** one signature (publisher). The consumer's consent is recorded as a local PoAC™ record, not as a returned signature.
- **org_internal, org_cross:** two signatures — initiator signs the core; the counterparty organization verifies and countersigns the identical core; only the doubly signed record constitutes an established relationship. Both parties retain the countersigned artifact as evidence. (Informative: this applies the PoAE™ philosophy to the relationship itself — a single-signature org handshake would prove only "A offered and some device on B's side went along"; countersignature proves both organizations authorized the relationship with their org keys.)
- **private_personal:** one signature; mutual establishment is symmetric-consent recorded on both clients.

VII-3.3 — Profile type confusion is structurally impossible. The profile field lies inside the signed core. There is no renegotiation, no downgrade, no reinterpretation. A profile change is a new handshake.

3.2 Extensions Container

VII-3.4 — The signed core and every grant object carry an extensions container:

```
extensions: [
  { ns: "optirando.attestation.dns", version: 1, critical: false, payload:
    {...} },
  { ns: "optirando.grant.ttl",          version: 1, critical: true,  payload: {...} }
]
```

VII-3.5 — Criticality semantics. Unknown non-critical extensions MUST be preserved and ignored — legacy handshakes never break. An unknown critical extension MUST cause visible refusal with the reason rendered ("requires extension ⟨ns⟩, unsupported"). A client MUST NOT strip, reorder, or partially process the container; it is covered by the core signatures and cannot be injected or removed by third parties.

VII-3.6 — Extension namespaces are registered in the Optirando™ extension registry (companion to the profile registry). Registration fixes ns, payload schema, versioning rules, and criticality rules. The container itself never changes.

VII-3.7 — Informative rationale. Pre-defined "empty fields" were considered and rejected: they force guessing the shape of unknown future content and still break when reality diverges from the reservation. The container defers shape decisions entirely while keeping legacy compatibility exact (the X.509/TLS extensions pattern). Everything in Sections 8–10 that is not core is specified as extension namespaces; the two genuine core commitments of this annex are the signature list and the container.

3.3 SSO Identity Binding

VII-3.8 — Handshake identity **MUST** be derived from, or cryptographically attested by, the SSO-established identity. A handshake without valid SSO binding **SHALL NOT** produce a usable relationship, declaration cache entry, or grant.

VII-3.9 — Recipient binding of all subsequent messages uses the handshake identity; no message-level substitution.

VII-3.10 — Identity guards in handshake and all ack/return paths **MUST** use the full identity guard, never a single-claim (sub-only) shortcut. (Informative: the resolved cross-SSO beap_ingest_ack defect class; recurrence in the capability layer would cause silent declaration-cache rejection across SSO boundaries.)

VII-3.11 — Publisher signing keys for Manifest Roots **MUST** be bound to the publisher's SSO identity by attestation in the registry namespace. Rotation is an attested event; pre-rotation roots retain evidentiary validity, rotated-out keys validate nothing new.

3.4 Reserved Extension Namespaces

VII-3.12 — The following extension namespaces are reserved in the extension registry as of this version; their payload schemas are defined in Annex IX or subsequent registry entries, and their reservation here fixes their existence and purpose without core change:

- **optirando.handshake.prior_ref** — escalation lineage: a new handshake cryptographically referencing a prior relationship of the same parties (Section 4.3; Annex IX). Identity continuity and history clustering only; confers no rights.
- **optirando.credential.*** — post-establishment **Credential Attachments**: append-only, individually signed records referencing the handshake by content hash, carrying additional credentials of the same SSO identity (envelope schema per Annex IX). Attachments inform policy evaluation and rendering; they **NEVER** mutate profile, grants, or rights — those change only by consent (VII-2.4).
- **optirando.transport.*** — transport profile references per Annex IX (relay-default, direct P2P opt-in, High-Assurance transport tiers).

(Informative: the extensions container of Section 3.2 is covered by the core signatures and is therefore an establishment-time structure; credentials arising after establishment structurally cannot enter it. Credential Attachments are the append-only complement — a second signature chain of the same SSO identity, referencing the immutable handshake without modifying it. Establishment-time

credentials live in the container; post-establishment credentials live in the attachment stream; verification state over either lives in neither, but in local assessment records per VII-8.9.)

4. Handshake Profiles

4.1 Profile Registry

VII-4.1 — Every handshake declares exactly one profile. Profiles are registered in the Optirando™ profile registry with: identifier, version, mandatory/forbidden core fields, permitted ingress paths, role symmetry, required signature cardinality (VII-3.2), permitted grant vocabularies (Section 10.4), and attestation requirements (Section 8). New profiles are additive registry entries; the handshake machine is never modified for them.

VII-4.2 — An unknown profile, or a known profile at an unsupported version, causes visible refusal (analogous to VII-3.5 / manifest schema rule).

4.2 Profile Table (initial registry content)

Profile	Symmetry	Ingress	<code>publisher_attestation</code>	Signatures	Follow-on grants
<code>pbeap_publisher</code>	asymmetric	public WR Code®, BEAP invitation, relay code claim, Optirando Code Entry	mandatory (Section 8)	1	delivery rights, standing action scopes (Section 10)
<code>private_personal</code>	symmetric	red WR Code®, direct invitation	forbidden (VII-4.5)	1 + mutual consent records	full BEAP™
<code>org_internal</code>	symmetric	red WR Code® (org-bound)	optional per org policy	2 (countersigned)	full BEAP™
<code>org_cross</code>	symmetric	red WR Code® (bilaterally bound)	optional per bilateral policy	2 (countersigned)	full BEAP™

VII-4.3 — In `pbeap_publisher`, the publisher is by definition an organization; a path for purely private (natural-person) offerings does not exist in this profile.

VII-4.4 — Symmetric profiles (`private_personal`, `org_internal`, `org_cross`) establish full bidirectional BEAP™ per the Canon core; this annex constrains them only where explicitly stated (signatures, red codes, optional attestation).

VII-4.5 — In `private_personal`, the `publisher_attestation` block is forbidden, not merely optional: private users must be structurally incapable of carrying location/identity attestation data in handshakes, so the field can never be "politely requested" by a counterparty. Consequently, inbound geofencing policies never apply to private persons (Section 9.5) — geofencing filters publishers, never people.

VII-4.6 — Ingress-path neutrality. WR Code® scan, initial BEAP invitation message, relay code claim, and Optirando Code Entry are routes to the identical handshake request. The path is logged (PoAC™-adjacent) but **MUST NOT** change handshake semantics, mandatory fields, or resulting rights. One handshake implementation, not per-path dialects.

4.3 Profile Immutability and Escalation

VII-4.7 — There is no profile upgrade, conversion, or migration path. In particular, pbeap_publisher never converts to any symmetric profile (VII-12.1). A party desiring a different relationship type initiates a new handshake under the new profile, with new human authorization and, where required, countersignature. Prior relationships and their graph regions are unaffected. The new handshake **MAY** carry the optirando.handshake.prior_ref extension (VII-3.12) referencing the prior relationship, providing identity continuity and history clustering (VII-11.2); the new relationship's rights derive exclusively from its own consents — nothing is inherited, restored, or reanimated by the reference.

4.4 Red WR Codes®

VII-4.8 — Red WR Codes® are the ingress form for internal profiles. Two mechanisms with distinct roles, both mandatory:

- **Color coding** is the human affordance: red signals internal — do not forward, do not photograph, do not publish. It carries no security property.
- **Payload sealing** is the actual control: the code payload is sealed (X25519/HKDF/AES-GCM per the sealed-capsule primitives) to the organization's keys (org_internal), to both organizations' keys (org_cross), or to the personal identity (private_personal). An outside scanner obtains ciphertext.

VII-4.9 — Informative: a red code photographed in the background of a public photo is therefore a hygiene incident, not a security incident. Implementations and documentation **MUST NOT** present the color as the protection mechanism.

4.5 Account Classes and Publishing Entitlements

Handshake profiles are additionally gated by the initiating party's account class. Four classes exist: Free, Pro, Publisher, Enterprise.

VII-4.10 — Offering entitlements per class:

Account class	May initiate as offering/organizational party	May initiate private_personal	Consumer role
Free	—	yes (red WR Code®, direct invitation)	unrestricted
Pro	—	yes	unrestricted
Publisher	pbeap_publisher, org_internal, org_cross	yes	unrestricted
Enterprise	pbeap_publisher, org_internal, org_cross	yes	unrestricted

VII-4.11 — Free and Pro accounts **MAY** create internal (red) WR Codes® for private_personal handshakes and **MAY** send BEAP™ messages over any established handshake. They **MUST NOT**

publish pbeap_publisher offerings and MUST NOT initiate organizational profiles. This is consistent with VII-4.3: a pBEAP™ publisher is by definition an organization.

VII-4.12 — The consumer role is account-class-independent. Any account class may scan public WR Codes®, establish pbeap_publisher handshakes as consumer, hold and revoke grants, and consent to prepared actions. Entitlements gate offering, never receiving — the protection model of Sections 9–10 belongs to every user.

VII-4.13 — Enforcement anchors. The entitlement is enforced at two independent points: (a) the registry — publisher key attestation (VII-3.11) and registry namespaces are issued exclusively to Publisher and Enterprise accounts, so a non-entitled account cannot produce a verifiable Capability Declaration or attain any attestation level above nothing; and (b) the authoring surface — publishing and organizational-profile controls are absent for non-entitled accounts (VII-15.8), following the structural-absence pattern of VII-15.2. Client-side checks alone are never the enforcement.

VII-4.14 — Informative, implementation staging: the Enterprise class is specified here for wire-format and registry completeness but is excluded from the MVP; Enterprise onboarding is realized in a later stage via Silicon Cargo. Nothing in this annex depends on Enterprise availability: Publisher-class entitlements are a strict subset surface, and the profile registry requires no change at Enterprise activation.

4.6 Initiation Classes (Orthogonality)

VII-4.15 — Initiation classes (Annex IX registry: public_bearer, targeted_bound) are orthogonal to handshake profiles. An initiation class selects **how a handshake request comes into being** — the invitation, parking, claim, and consent mechanics preceding establishment — while the profile fixes **what the established relationship is**. A fast public initiation (public_bearer) always establishes pbeap_publisher; full-context initiation establishes the symmetric profiles per Section 4.2. No initiation class alters profile semantics, mandatory fields, signature cardinality, or resulting rights (consistent with VII-4.6). The terms "bidirectional handshake" and "full-context handshake" refer exclusively to the symmetric (org) profiles established through the full roundtrip — never to any pBEAP™ relationship, which remains asymmetric without exception (VII-12.1). Initiation classes, their invitation objects, and their claim protocol are normatively defined in Annex IX; this annex governs everything from the handshake request onward, unchanged.

5. Capability Declaration — Variants A and B

5.1 General

VII-5.1 — The capability declaration phase follows the identity phase and the context roundtrip, is sealed with the same primitives (relay remains a blind courier), and exists in two variants. Both variants declare the same leaf structures under the same verification chain (Section 7) — two transport moments, one trust model, never two. The declaration is carried in the handshake core's declarations container under the registered namespace optirando.decl.capability (VII-3.1); the structures specified in this section are that namespace's payload.

VII-5.2 — Handshakes without a declaration (legacy / symmetric profiles without offerings) remain valid; subsequent ingress then requires online registry verification (level R) before any Prepare.

5.2 Variant A — Direct-Load

VII-5.3 — When the ingress artifact (scanned WR Code®, invitation message) identifies exactly one offering, the handshake request MAY carry that offering's leaf hashes and inclusion proofs inline, in addition to the Manifest Root. After consent, the client immediately fetches from the public registry, completes the full Section 7 chain, anchors into the target region, and the system is usable. Scan → consent → use.

VII-5.4 — Variant A never carries more than the one offering identified by the ingress artifact (VII-2.1 catalog-blob prohibition).

5.3 Variant B — Deferred Capsule

VII-5.5 — The handshake carries only the Capability Declaration Block; leaves follow later via roundtrip capsules or message-level ingress. The block contains:

Field	Content
provider_id	SSO-bound provider identity (matches the identity phase, VII-3.9)
registry_ns	Provider's public registry namespace
manifest_root	Merkle root over all published commitments
root_sig	Signature over manifest_root root_seq issued_at (provider key per VII-3.11)
root_seq	Monotonic root sequence number
issued_at	Timestamp of root issuance
base_models	Exact base-model identities the provider publishes adapters for (Section 6.1, never friendly names alone)
bridge_avail	Informative hint whether bridge adapters exist and for which pairs; resolution is always manifest-level (Section 6.3)
decl_schema	Schema version of this block; unknown fields preserved-and-ignored (VII-6.1.5 rule)

VII-5.5.1 — Root rotation. Rotation revokes leaves: a consumer holding root seq=n who receives a declaration with seq=m > n MUST replace the cached root; inclusion proofs are evaluated against the newest known root; providers MUST republish still-valid leaves under each new root.

VII-5.5.2 — Offline root aging. In field operation modes without registry reachability, a cached root remains usable for at most root_max_age after issued_at (deployment parameter, normative default 30 days). On expiry, the Capability Diff renders verification level H-expired (Section 7) and the Prepare step for new acquisitions is blocked until a registry re-check succeeds; already-acquired, PoAC™-recorded capabilities remain usable.

5.4 Fast-Path Consent

VII-5.6 — The two-step Prepare/Authorize commit (Section 14.4) MAY collapse into a single human action if and only if everything is green: all applicable verification levels attained, publisher attestation meets local policy minimum, no bridge resolution involved, no capability gaps, no data obligations beyond those rendered on the consent screen, and no standing-scope grant bundled (VII-10.5.1). The single consent screen IS the complete Capability Diff — one button because there is nothing to decide, never because something was hidden.

Under the fast public initiation class (public_bearer, Annex IX), the Fast-Path collapse carries one additional mandatory condition: the consent SHALL sign over the hash-pinned first-artifact

reference carried by the invitation, and the first BEAP™ interaction SHALL match the consented hash — a mismatching first payload is rejected (Annex IX). All conditions of this section and VII-5.7 apply unchanged in addition.

VII-5.7 — Any gap, bridge, schema mismatch, attestation shortfall, or standing-scope request forces the full two-step flow. Fast-Path is a rendering optimization of consent, not a reduction of it.

6. Adapter Manifests, Canonical Naming, Bridge Adapters

6.1 Adapter Manifest

VII-6.1.1 — Every adapter (including every bridge adapter) MUST be accompanied by a signed Adapter Manifest. Loading, compatibility, and anchoring decisions consume manifest fields exclusively (VII-2.2).

VII-6.1.2 — The manifest exists in two copies that MUST be byte-identical: (a) embedded in the safetensors metadata header, enabling offline field-mode verification against the artifact bytes alone, and (b) a registry copy in the provider namespace, enabling discovery and pre-fetch decisions without downloading artifacts. Divergence between the copies is a hard verification failure (VII-14.6).

VII-6.1.3 — Manifest fields:

Field	Requirement	Content
manifest_schema	MUST	Integer schema version, starting at 1
artifact_class	MUST	lora_adapter bridge_adapter
base_model	MUST	Exact base-model reference: registry ID or content hash of the base weights the adapter was trained against — never a friendly name alone. Fine-tune variants of a nominal model are distinct base models.
base_model_display	SHOULD	Human-friendly form for UI and name derivation
geometry	MUST	Rank, alpha, target modules, tensor format/schema version (PEFT/safetensors), quantization of intended deployment
scope	MUST	Task/domain anchor consumed by graph anchoring (Section 11): machine class, automation class, or region descriptor, e.g. deepfix/hydpres-9000/diagnostics
bridge_profiles	MUST (empty allowed)	List of named bridge-conformance profiles this adapter declares. Empty at launch; populated as open bridge standards emerge. Providers MUST NOT declare unfalsifiable blanket compatibility.
training_artifacts	SHOULD	Commitments enabling honest future bridging: tokenizer alignment info, calibration/eval set hash
bridge_pair	MUST for bridge_adapter, absent otherwise	Ordered pair (source base_model, target base_model) plus implemented profile list
provider_id, sig	MUST	Provider identity and signature over the canonical manifest serialization

VII-6.1.4 — `base_model` mismatch is never silently tolerated. If a required adapter's `base_model` does not match the locally deployed base, the only permitted resolutions are (a) bridge resolution per Section 6.3, rendered explicitly in the Capability Diff, or (b) refusal.

VII-6.1.5 — **Schema evolution.** Unknown manifest fields **MUST** be preserved and ignored, never rejected. A manifest with `manifest_schema` greater than the client's supported version **MUST** fail visibly in the Capability Diff ("requires manifest schema N, unsupported"), never silently misload.

VII-6.1.6 — `bridge_profiles` declarations are falsifiable claims: declaring a profile asserts conformance to that profile's published test criteria, cryptographically accountable through the signature chain.

6.2 Canonical Naming

VII-6.2.1 — The canonical name of an artifact is deterministically derived from its manifest and is display and filesystem convenience only — normatively non-authoritative:

`{scope-short}-{base-family}-{size}-{quant}-r{rank}[-br{n}]`

e.g. `hydpres9000diag-orion3-12b-q4-r16`

`bridge-orion3to4-12b-br1`

VII-6.2.2 — Implementations **MUST NOT** parse canonical names to make loading, compatibility, or anchoring decisions.

VII-6.2.3 — A derived-name/manifest mismatch (the name does not regenerate from manifest content) is an integrity signal surfaced in the Capability Diff provenance strip; it does not by itself block loading if the Section 7 chain verifies, but it **MUST** be visible.

6.3 Bridge Adapters

VII-6.3.1 — A bridge adapter is an ordinary artifact under this annex: manifest, Merkle leaf, provider signature, own provenance rendering. A bridge is a third party's artifact entering the trust chain and receives its own verification strip in the Capability Diff — never inherited trust from the automation that occasioned its use.

VII-6.3.2 — **Resolution.** When a required adapter X declares `base_model = A` and the local deployment runs base $B \neq A$, the client queries verified manifests for a bridge entry with `bridge_pair = (A → B)` whose implemented profiles intersect X's `bridge_profiles`. An empty intersection — at launch, always, since `bridge_profiles` starts empty — means no bridge resolution exists and the mismatch renders as an unresolvable capability gap.

VII-6.3.3 — Bridge resolution is never automatic: the Capability Diff renders "compatible via bridge adapter" as a distinct, opt-in resolution requiring the full Prepare/Authorize flow, with a reduced-fidelity flag displayed and recorded, and the bridge's own provenance expanded before Prepare can be confirmed.

VII-6.3.4 — Capability acquisition via bridge produces a PoAC™ record identifying the original adapter, the bridge adapter, both manifest hashes, the profile under which resolution occurred, and the reduced-fidelity flag.

7. Verification Chain and Levels

VII-7.1 — The complete chain, all links mandatory before host-side staging of any artifact:

SSO-attested provider key (VII-3.11)

- signature over Manifest Root (root_sig, root_seq)
- Merkle inclusion proof (leaf = H(artifact_hash || manifest_hash))
- artifact hash (fetched or received bytes)
- local bytes (recomputed, compared)

+ embedded manifest == registry manifest (VII-6.1.2, where registry reachable)

Registry hash agreement alone never authorizes loading (VII-2.3). (Informative: a compromised registry or MITM'd fetch that could satisfy a hash-lookup check would otherwise inject adapters directly into the host trust root. Safetensors is execution-safe by format, but a poisoned adapter is a behavioral attack — e.g., exfiltration via generated BEAP™ payloads, directly relevant to the journalist/NGO deployment profile. The signature chain does not prevent poisoning; it makes the provider cryptographically accountable for every byte that shaped model behavior.)

VII-7.2 — Artifact verification levels, discrete and individually rendered: S (signature) · H (proof vs. cached root) · H-expired · R (registry-confirmed) · L (ledger-anchored, future). Composable (e.g., S+H+R online; S+H offline field mode).

VII-7.3 — Post-verification routing: adapters MAY load on host; tools sandbox-routed unconditionally.

8. Publisher Attestation

Specified as extension namespaces (optirando.attestation.*); mandatory for pbeap_publisher (VII-4.2), structurally forbidden for private_personal (VII-4.5).

8.1 Two-Entity Model

VII-8.1 — The publisher_attestation block distinguishes two entities whose data domains overlap but are not identical:

- **domain_anchor** — the entity controlling the domain and named in the website's legal notice (Impressum): legal name, registered address, jurisdiction, register number, domain. This is what DNS+WebPKI can actually verify.
- **offering_entity** — the legal person offering the handshake and contracting the automation: same field set, own jurisdiction. This is who the user actually needs to know.

(Informative: the canonical example — domain and legal notice (Impressum) held by Northbay Beverages Inc. (US), offering entity Northbay Beverages Deutschland GmbH (DE), a subsidiary contracting automations for the German market under the parent's domain.)

VII-8.2 — Both jurisdictions are independent, separately attested fields. Divergence is not a warning per se — multinational structures are the normal case — but it is always rendered (VII-14.2).

8.2 Delegation

VII-8.3 — The link between the entities is a delegation: the domain_anchor publishes, under its domain (in the well-known document, Section 8.3), a signed list of authorized offering_entity records — key fingerprints and declared relationship (same / subsidiary / affiliate / authorized_agent). The claim "entity E offers under domain D" is thereby countersigned by the domain holder and verified by the same P1 machinery; no new infrastructure.

VII-8.4 — Without a delegation entry, the offering_entity remains at P0 even where the domain attains P1, and the UI renders exactly this asymmetry ("domain verified (US) — offering entity declared only (DE)"). No silent error.

8.3 Verification Levels P0/P1/P2

VII-8.5 — Attestation levels are discrete, ordered, and rendered separately from artifact levels (second provenance row, VII-14.2):

- **P0 — declared.** Self-asserted, signed. Proves authorship of the claim only.
- **P1 — DNS+WebPKI corroborated.** Two independent channels MUST agree on the identical attestation block:
 1. **DNS:** TXT record at _optirando.<domain> carrying the attestation-block hash, publisher key fingerprint, issued_at, rotation interval. DNSSEC, where present, raises recorded confidence but is not a precondition.
 2. **WebPKI:** https://<domain>/.well-known/optirando/attestation.json, served under a valid TLS certificate for the domain, identical block. An attacker must simultaneously forge DNS answers and hold a valid certificate — two separate compromises. A verification_perspectives field is present from v1 (multi-vantage querying per the ACME pattern as later hardening; initial value MAY be 1). Corroboration ages analogously to root_max_age; expiry degrades visibly to P1-expired, never silently to P0.
- **P2 — externally attested (later):** register reconciliation, ledger anchoring, hosting-reality checks. Pure extension; schema prepared, no wire change.

VII-8.6 — **Honest semantics, normative wording requirement:** P1 proves domain control at verification time plus consistency of the claim — never the physical truth of the address. Physical/registry truth begins at P2. UI copy and documentation MUST NOT overstate P1.

VII-8.7 — **Informative rationale (the anti-dark-office argument):** the dominant attack economics rely on offshore anonymity ("dark offices"). P1 does not verify addresses; it forces the attacker to control a real domain carrying a real, publicly liable Impressum, binding the attack to a jurisdiction and a cost structure. Enforcement beyond this (the technically complex part) arrives later as P2+ mechanics without schema change — exactly the modular-extension requirement.

VII-8.8 — Private-Distribution-Path publishers carry the same mandatory attestation: the Private Distribution Path keeps automations non-public, never publisher identity (Section 13.2).

VII-8.9 — **Verification state as assessments.** Attestation levels are local assessments computed — and re-computed — by the consumer's verifier over the attestation credential; they are never a stored property of the handshake, the publisher record, or the credential itself. Verification state SHALL be maintained as separate, append-only assessment records — level, subject reference

(attestation-block hash), evidence references, verified_at, valid_until, verifier identity — per the assessment model of Annex IX. New verification mechanisms (P2 and beyond) are additional verifier modules emitting assessments of identical shape; policy engines (Section 9) and rendering surfaces (Section 14) consume current assessments and never persist levels elsewhere. Expiry is therefore visible by construction (P1-expired is simply an assessment whose valid_until has passed), and a publisher attaining a higher level upgrades the rendering of every existing relationship without any handshake mutation.

9. Handshake Admission Policy (Geofencing)

VII-9.1 — Optirando™ supplies the mechanism (verified jurisdiction data in discrete levels); the admission decision is made by a local, org- or user-controlled policy engine. There is no platform-level admission decision. (Informative: freedom of choice, not paternalism — a user must know whom they are dealing with; the org must be free to decide with whom it shakes hands.)

VII-9.2 — The policy engine evaluates, at minimum: allow/deny over domain_anchor.jurisdiction and offering_entity.jurisdiction (configurable: either-suffices / both-must-pass), and a minimum attestation level per profile (e.g., "pbeap_publisher below P1 is not rendered").

VII-9.3 — **Sanctions exclusion** is implemented as importable policy sources: the org subscribes to lists (EU, OFAC, custom). The system hard-codes no country list.

VII-9.4 — **Normative default recommendation:** meaningful geofencing presupposes min_attestation: P1 for pbeap_publisher — sanctions filtering over P0 (self-declared) data is near-worthless, since a sanctioned party will not declare honestly. Deployments enabling jurisdiction policies SHOULD enforce P1 minimum.

VII-9.5 — Admission policies never evaluate private_personal counterparts (no attestation exists to evaluate, VII-4.5). Geofencing filters publishers, never persons.

VII-9.6 — Every policy refusal produces an admission record (PoAC™-adjacent) and is rendered to the user with the matching rule — the user's own policy is not exempt from the no-silent-error principle.

10. The Grant Model

This section governs every right a publisher can hold toward a consumer. It is the normative heart of this annex.

10.1 The Execution Boundary

VII-10.1 — **No grant confers execution authority. Ever.** Grants are limited to (a) **delivery rights** — the right to have transmissions accepted by the consumer's client, and (b) **preparation rights** (standing action scopes) — the right to have payloads staged as prepared actions. Every execution of any action, however prepared, is a distinct human act (a consent tap) producing a PoAE™ record. Preparation makes consent cheap — never unnecessary.

(Informative: the strength of BEAP™ in the recurring-interaction case — e.g., a medical practice delivering appointments — is not that consent disappears, but that it shrinks to one second because verification, staging, and effect derivation happened before the human was asked. This is preparation ahead of use — the WR Preparation stages (Annex VI) — applied to the publisher

channel. The resulting UX is also more honest than auto-execution: the user sees the appointment and confirms it, rather than later discovering a calendar entry they never touched.)

10.2 Delivery Rights

VII-10.2 — A delivery right is scope-bound ("stay-related messages", "appointment notifications"), enforced at the consumer's client (VII-2.7): transmissions outside the declared scope are blocked before visibility and logged; repeated off-scope delivery is surfaced with a one-tap revoke offer.

VII-10.3 — Every delivered item carries a reference to the grant under which it is delivered; the client renders this provenance ("delivered under: appointment notifications, granted 2026-07-11").

VII-10.4 — Consumer replies remain request-envelope-conformant (Section 12): a delivery right gives the publisher messages and payloads to pinned templates — never free templates. The role asymmetry of pBEAP™ survives every grant.

10.3 Standing Action Scopes

VII-10.5 — A standing action scope pins:

- the exact template leaf hash (full Section 7 chain at grant time; graph-anchored in the publisher region),
- a typed effect vocabulary entry (VII-10.4.1),
- a mandatory payload schema (part of the pinned manifest),
- constraints: target resource (a specific calendar, a designated inbox region — never a resource class), permitted data classes, rate.

Under the scope, the publisher delivers payloads. The client verifies each payload inbound — leaf reference against the pinned scope, payload against the schema, content against the constraints — anchors it, and renders a prepared action with its effect displayed precisely ("Practice X: appointment Aug 14, 9:30 → add to calendar Y"), the effect being deterministically derivable from the verified template. One tap executes (PoAE™); no tap, nothing happens.

VII-10.5.1 — Unbundling. A standing action scope is NEVER part of a Fast-Path consent (VII-5.6). It is always a separate consent step with its own screen showing template identity, effect, constraints, and revocability. (Informative: the foreseeable failure is not the hacker but the publisher bundling standing rights into the check-in scan's fine print. Structural separation makes that impossible.)

VII-10.5.2 — Anti-nagging. Declining a scope MUST NOT block the primary service: the scope prompt renders after completion of the primary flow, never as its precondition — this is additionally a normative publisher obligation, so violations are at least documented violations. Scope requests are rate-limited per publisher identity; a declined request does not reappear on every subsequent touchpoint scan.

VII-10.5.3 — Template pinning. A template version change is never a payload; it is a capability change → Capability Diff with version delta, new consent. A publisher cannot broaden their own authority by update.

VII-10.5.4 — Payload validation (MUST). Payloads are validated strictly against the pinned schema before rendering, not merely before execution: an out-of-schema payload is blocked and logged like a constraint violation. Purpose: a manipulated payload must be unable to mislead the

consent screen or cause harm at render time — the consent screen's claimed effect and the tap's actual effect are guaranteed identical (the anti-silent-error property of the whole construction). A restricted declarative template language for standing-scope templates is registered as a later hardening (profile vocabulary), not required in this version.

VII-10.5.5 — Batching. The client MAY offer accumulated prepared actions for collective confirmation ("add 5 appointments?") — consent over a visible set. The UI MUST NOT offer any auto-accept toggle ("always execute from this publisher"): that would reintroduce execution authority through the back door.

10.4 Vocabularies

VII-10.4.1 — Standing action scopes exist only for named effect classes in the Optirando™ effect vocabulary registry (initial entries: calendar entry, document deposit into a designated inbox region, notification). The registry grows additively (register pattern); "arbitrary execution" is not and never becomes a vocabulary entry. Effects without a vocabulary entry remain envelope-bound with per-instance consent.

VII-10.4.2 — Each profile declares which grant vocabularies it permits (VII-4.1). Symmetric profiles are unconstrained by this section beyond VII-10.1, which is universal.

10.5 Grant Creation and Modification

VII-10.6 — Grants are created only on an explicit consent screen rendering: publisher identity + attestation level, grant type, scope, constraints, any agreed limits (Section 10.7), and revocability. Grant creation and every modification (scope broadening, template repinning, limit change) is a PoAC™ event. Payload deliveries under a scope are not PoAC™ events; executions are PoAE™ events referencing the grant.

10.6 Relationship Lifecycle

The lifecycle is: **scan** → **consent** → **use** → **revoke** → **(new scan, any time)**. Relationships are cheap, disposable, and cheaply re-establishable objects; the model's spam resistance rests on the pairing of cheap unilateral revocation with cheap re-establishment, not on automatic expiry.

VII-10.7.1 — Default: no expiry until revoke. A grant without an agreed limit is an unlimited grant — this is the well-defined ground state of the wire format, not an absent field. Rights are state-stable: no automatic teardown, no idle timeout, no system-initiated disconnection exists. (Informative: automatic expiry would itself be a silent state change — the user granted a right and the system takes it back with nobody having decided. The hotel guest receiving the invoice or a lost-property message three days after checkout is the practical proof.)

VII-10.7.2 — Revocation is unilateral, immediate, and silent. Revocation kills the delivery right and all standing scopes of the publisher, is enforced client-side without publisher cooperation, and generates no notification, bounce, or signal to the publisher — their transmissions simply die before visibility, indistinguishable from the user ignoring prepared actions. The user owes no explanation, and no revocation side-channel exists.

VII-10.7.3 — Revocation severs rights, not history. The publisher region in the consumer's WR Graph™ — sessions, PoAE™/PoAC™ records, contents — persists. A later re-handshake clusters into the same region via the attested publisher identity: history continuous, rights discontinuous.

(Informative: the decisive advantage over email, where the archive is married to the open channel; here the archive is kept and the channel closed.)

VII-10.7.4 — Re-handshake reanimates nothing. A new handshake after revocation starts with empty rights. Previously held scopes MAY be shown as an explicit restore proposal ("previously granted: prepare calendar entries — grant again?") but never revive automatically; otherwise revocation degrades to a pause the publisher can speculate on.

10.7 Agreed Limits (Advanced)

Optional, self-consented lifetime limits, specified as grant extensions. Both are advanced options: the standard consent shows no lifetime line at all (nothing to show in the ground state); limits live behind an expandable advanced section, with unlimited-until-revoke as the untouched default.

VII-10.8.1 — `optirando.grant.single_use` (critical). The handshake carries exactly one prepared action; its consented execution (the PoAE™ record) dissolves the relationship. Trigger is execution, never delivery: the action may be delivered and left untouched indefinitely — an ignored single-use handshake persists until manually revoked (consistent with VII-10.7.1), unless combined with `ttl` (combination permitted). (Informative: a strong primitive for payments, one-time forms, guest access — the publisher is structurally unable to follow up, and the user knows it at scan time.)

VII-10.8.2 — `optirando.grant.ttl` (critical). Dissolution X days after grant. Expiry is mechanically a revocation with a pre-consented timestamp: the three lifecycle pins (VII-10.7.2–.4) apply unchanged, including silence toward the publisher.

VII-10.8.3 — Criticality direction. A client not understanding a present limit extension MUST NOT accept the grant as unlimited; it refuses visibly (VII-3.5). A client finding no limit extension has the normal case — the criticality rule guards only against misreading a limit, never against the default.

VII-10.8.4 — Dual declarability, user decision. The publisher MAY propose a limit in the offering (a one-time check-in code sensibly ships `single_use`); the user MAY set or tighten limits in the advanced consent options. The stricter declaration wins. The user cannot loosen a publisher-declared limit — the publisher's offer is the ceiling of granted rights; more requires a new handshake.

VII-10.8.5 — Agreed limits are rendered at grant time ("single action" / "expires 2026-07-25") — they are part of what is consented — and continuously in the relationship overview.

11. Sessions, Hierarchy, and Identity Clustering

VII-11.1 — Sessions are organizational and display constructs in the WR Graph™; they carry scope, context, and hierarchy. Sessions do not govern grant lifetimes (VII-10.7.1): the graph organizes, grants authorize, and the two layers are decoupled.

VII-11.2 — Identity-keyed clustering, local and verified. Clustering of touchpoints occurs on the consumer's side, keyed on the attested publisher identity — never on a publisher claim. Same attested key ⇒ same publisher region.

VII-11.3 — Hierarchy. An ingress artifact MAY carry a session reference to a parent session (room code referencing the stay session; machine session under the site session — the pattern is publisher-generic, not hotel-specific). The child anchors under the parent if and only if the publisher identity

matches and the consumer holds the referenced session and the reference verifies. Otherwise it renders as a new, independent session — never silently merged.

VII-11.4 — The parent session provides collective display and collective revocation affordances ("Hotel X — your stay": check-in, room 412, spa; each with own scope, jointly revocable via the parent). Grants attach to the publisher region, not to individual sessions: a second touchpoint under the same attested identity extends nothing and multiplies nothing — one grant remains one grant.

VII-11.5 — Graph anchoring. Verified artifacts are linked into the WR Graph™ region designated by the manifest scope and the ingress context — for machine automations, the machine-session region (Annex IV). A twenty-machine provider yields per-machine regions each carrying exactly its own adapters and templates; no catalog-wide linking occurs. Ingress referencing a not-yet-existing region creates it as part of Prepare, recorded in the PoAC™ acquisition record.

VII-11.6 — Capability ingress with resolvable gaps is a valid preparation trigger in the sense of the WR Preparation stages (Annex VI): the system MAY pre-stage fetch and verification plans upon Capability Diff display, but MUST NOT fetch or link before Prepare is authorized (VII-2.4, VII-14.4).

VII-11.7 — Every adapter acquisition, template import, bridge resolution, and region creation under this annex produces a PoAC™ record per Annex III, carrying: leaf hash(es), manifest hash(es), verification level attained, ingress path, target region, and — for bridges — the VII-6.3.4 fields. Imported templates carry their provenance (publisher, leaf, envelope) as region metadata and remain separated from local session history.

12. pBEAP™ — Protocol and Invariant

VII-12.1 — The pBEAP™ invariant (final normative form):

pBEAP™ establishes a **peer-to-peer relationship that persists until revocation** — one in which the publisher never holds execution authority, roles never symmetrize, and nothing escalates silently. Grantable rights within the relationship are exclusively delivery and preparation rights — typed, pinned, constraint-bound, unilaterally revocable. Every execution is a human-consented act with a PoAE™ record; preparation makes it cheap, never unnecessary. Revocation ends the relationship's rights at the consumer's sole discretion; repeated contact after revocation means repeated consent via a new handshake.

(Informative: persistence, role symmetry, and transport are three independent axes of a relationship, and pBEAP™ fixes them as follows. **Persistence** is the point: the consumer uses the P2P relationship for its purpose — a hotel stay, an ongoing practice relationship — and revokes when done; until then it stands (VII-10.7.1). **Role symmetry** is permanently excluded: a conversion of an established pBEAP™ relationship into a symmetric channel with free bidirectional template exchange was considered during design and rejected — a standing publisher-initiated symmetric channel would be the structural spam opt-in at which the model collapses. A party desiring a symmetric relationship initiates a new handshake under a symmetric profile (VII-4.7). **Transport** is the sealed coordination relay (blind courier); delivery rights and standing scopes are pure receiver-side admission policy requiring no transport change.)

VII-12.2 — Role asymmetry. The offering party MAY attach templates only inside a request envelope, or deliver payloads to consumer-pinned templates under a standing scope. The receiving party MAY return requested data and PoAE™ logs and MUST NOT attach templates in any pBEAP™ response. Enforced structurally in the composer (VII-15.2).

VII-12.3 — Request envelope. The pBEAP™ request envelope contains: the request purpose, the data fields requested, attached template references (leaf hash + inclusion proof, or inline templates hash-bound to leaves), required adapter references for the specific automation, the data obligations declaration (VII-12.4), and envelope_schema (preserve-and-ignore evolution per the VII-6.1.5 rule).

VII-12.4 — Data obligations declaration. Every envelope and every standing scope declares machine-readably and human-renderably: requested data (e.g., PII fields), requestable PoAE™ logs, and every outbound data flow any involved template performs, with destination endpoints. Rendered as the Data Obligations section of the Capability Diff / consent screen. Actual flows exceeding the declaration are a hard failure (VII-14.6).

VII-12.5 — Backchannel authorization. Where a template performs outbound insertion of consumer data into the publisher's system (the PII-backchannel pattern), the execution consent screen displays destination and the exact data leaving; the PoAE™ record attests that the consumer authorized that data to that endpoint, never merely "template executed". (With VII-10.1 this is now simply the general rule — every execution is consented — with an enhanced display obligation for outbound flows.)

VII-12.6 — Within pBEAP™, adapters are loaded exclusively from the public Optirando™ repositories (or self-hosted endpoints on the Private Distribution Path) under the Section 7 chain — under Variant A immediately post-consent, under Variant B on message-level ingress. Nothing about the sealed relay's blind-courier property changes for any grant type: delivery rights and standing scopes are pure receiver-side admission policy, requiring no transport change.

13. Distribution Paths

VII-13.1 — Public path. Registry namespace hosts root history, manifests, artifacts, key attestation; fetch over verified HTTPS with mandatory hash display (curated lists remain rejected as inappropriate for the threat model).

VII-13.2 — Private Distribution Path. Artifacts self-hosted; registry stores only the signed per-provider root (one root covers public and non-public offerings alike; per-automation commitments prohibited — cardinality leaks offering structure). Evidentiary reveal: leaf + proof chains to the timestamped public root. Publisher attestation remains mandatory (VII-8.8).

VII-13.3 — Ledger anchoring (planned). Roots only, at rotation events; adds level L; alters no other mechanism.

14. UI Specification

14.1 Capability Diff

The Capability Diff is the single reusable surface answering the anti-silent-error requirement: the user received a capability claim and must see, before anything loads, the difference between what is required and what their system carries.

VII-14.1 — One component, all paths. The Capability Diff MUST render on every ingress path: handshake-declared offering, WR Code® scan, pBEAP™ request envelope, and BEAP™ messages received on any other channel. Path-specific dialects are prohibited.

VII-14.1.1 — Requirements vs. local state. Per required item, one row with a resolution state:

Item class	Possible states
Base model	match · mismatch (unresolvable) · compatible via bridge (reduced fidelity) — bridge row expands to its own provenance strip (VII-6.3.1)
Adapter	cached & verified · fetch needed · no compatible variant · schema unsupported (VII-6.1.5)
Template	verified · fetch needed · envelope-bound (pBEAP™)
Tool	present · missing — rendered with a visually distinct sandbox badge; tools never gain host routing regardless of state (VII-2.5)

VII-14.1.2 — Every state is understandable without documentation: plain-language labels, with underlying hashes and proof status exposed on demand for expert inspection.

VII-14.2 — Dual provenance rows. The provenance strip renders two separate rows: artifact verification (S/H/H-expired/R/L) and publisher attestation (P0/P1/P1-expired/P2), each level as a discrete attained/failed/expired indicator. A publisher with perfect artifact proofs but P0 attestation is precisely the case the user must see. Jurisdiction divergence between domain_anchor and offering_entity, and missing delegation (VII-8.4), render here.

VII-14.3 — Consent screens. Grant-creation screens per VII-10.6; standing-scope screens always separate (VII-10.5.1) and post-primary-flow (VII-10.5.2); agreed limits shown at grant time; the default (unlimited-until-revoke) shows no lifetime line (VII-10.8.5, ground-state silence); advanced limit options behind an expander.

VII-14.4 — Commit flow. Two-step commit:

- **Prepare** — resolves gaps: fetches artifacts, completes the Section 7 chain, stages into (or creates) the target region; a preparation execution in the sense of the WR Preparation stages (Annex VI), producing PoAC™ records for acquisitions.
- **Authorize** — activates the capability set for the region and, for envelopes with outbound flows, arms the VII-12.5 gate for execution time.

Nothing between ingress and an authorized Prepare touches the host trust root. Prepare and Authorize are separate human actions; combining them into one control is non-conformant — with the sole exception of the Fast-Path collapse under the VII-5.6/5.7 conditions.

VII-14.5 — Relationship overview (new, first-class surface). Because nothing cleans itself up (VII-10.7.1), the rights inventory grows monotonically until the user acts. The UI therefore provides a first-class relationship overview: all publisher regions with active grants, attestation level, last delivery, held scopes and limits, and one-tap revoke per row — not buried in settings, and additionally anchored where prepared actions arrive (persistent relationship status per publisher region). The revoke tap shows once what falls away (delivery right, n scopes), confirms, done — the "checkout gesture": leave the hotel, close the relationship, keep the history. The system MAY propose cleanup ("Hotel X, last delivery 14 months ago — revoke?") — proposal is preparation; the tap remains the authorization.

VII-14.6 — Hard-fail rendering. Hash mismatch, invalid signature, failed inclusion proof, embedded/registry manifest divergence, payload schema violation, constraint violation, off-scope delivery (blocked pre-visibility, logged, surfaced with revoke offer), declared-vs-actual flow divergence, unknown critical extension, unknown profile. Refusals, not warnings; no override control.

VII-14.7 — Relationship imprint rendering. The relationship overview (VII-14.5) and the per-relationship detail surface SHALL render the attestation block's imprint fields — for both `domain_anchor` and `offering_entity`: legal name, registered address, jurisdiction, register number, and domain — together with the attestation-level badge of VII-14.2. Rendering follows the **store-reference / render-current-assessment** rule: the handshake stores the attestation reference; the displayed level and badge reflect the current verification-assessment state at render time (VII-8.9) — re-corroboration and aging included, P1-expired rendered as such per VII-8.5 — never the establishment-time snapshot alone. A publisher's later attainment of a higher attestation level upgrades the rendering of every existing relationship without any handshake mutation; a lapsed corroboration degrades it equally without one. UI copy remains bound by the honest-semantics rule of VII-8.6.

15. Authoring Requirements (BEAP™ Composer)

Normative requirements on the authoring surface. These constrain any conformant composer implementation; they are product requirements, not implementation guidance.

VII-15.1 — Attachment classes. (1) Inline template, hash-bound; (2) leaf reference (hash + proof; registry or declared self-host endpoint); (3) pBEAP™ request envelope, composable only in the offering role; (4) standing-scope payload — composable only against an existing consumer-granted scope, validated against the pinned schema at authoring time (a publisher cannot even compose an out-of-schema payload without warning).

VII-15.2 — Structural role enforcement. In the receiving role of a pBEAP™ context, template attachment controls of every class are absent, not disabled.

VII-15.3 — No handshake catalog control. The composer offers offering-scoped Variant-A inclusion (the one offering a generated WR Code® identifies) and message-level references; no control exists that injects catalog-level adapter enumerations into handshakes.

VII-15.4 — Data obligations authoring. Envelope/scope composition requires completion of the obligations declaration; the composer SHOULD draft the outbound-flow list by static template inspection and MUST require author confirmation; an envelope whose template flows exceed its declaration is unsendable.

VII-15.5 — WR Code® generation. Public codes: provider id, `root_seq`, offering-scoped leaves + proofs (Variant A), scope, envelope reference, optional proposed grant limits (`single_use` for one-time codes) — offline-verifiable against a cached root. Red codes: payload sealed per VII-4.8 to the profile-appropriate keys; the composer enforces the sealing, the color is applied as affordance.

VII-15.6 — Private-Distribution-Path authoring. Marking an automation non-public excludes its artifacts from registry publication, records self-host endpoints in envelopes, and commits its leaves under the per-provider root identically to public leaves (Section 13.2).

VII-15.7 — Grant-limit proposals. The composer supports publisher-side limit proposals (VII-10.8.4) with the stricter-wins rule documented in the authoring UI.

VII-15.8 — Entitlement gating. Publishing surfaces (pbeap_publisher offering authoring, public WR Code® generation, organizational-profile initiation) are absent for Free and Pro accounts, not disabled (VII-4.10–4.13). Internal (red) WR Code® generation and messaging over established handshakes are available to all classes.

16. Resolved Design Decisions (record)

#	Question	Resolution	Ref
1	Manifest Root coverage	All artifact classes — adapters, templates, bridges — under one root; one anchoring story	VII-3.11, VII-5.5
2	Adapter hashes in handshake vs. message	Root in handshake, leaves in messages; Variant A carries one offering's leaves inline when the ingress artifact itself supplies granularity	VII-2.1, VII-5.3
3	Merkle leaf definition	H(artifact_hash manifest_hash) — bytes and declaration jointly non-substitutable	VII-7.1
4	Manifest placement	Embedded and registry copy, byte-identical; divergence is a hard failure	VII-6.1.2
5	Offline root aging	root_max_age, default 30 days; H-expired blocks new Prepare, acquired capabilities remain	VII-5.5.2
6	Private-Distribution-Path commitment granularity	Per-provider; per-automation commitments prohibited (cardinality leaks offering structure)	VII-13.2
7	Bridge compatibility semantics	Named profile list, empty at launch, falsifiable claims only; no blanket declarations	VII-6.1.3, VII-6.1.6
8	Future-proofing mechanism	Extensions container with criticality flag; "empty reserved fields" rejected	VII-3.4–3.7
9	Signature cardinality	Signature list 1..n in core; org profiles require countersignature; pBEAP one signature + local consent record	VII-3.2
10	Adapter delivery UX	Two declaration variants: A direct-load (offering-scoped inline leaves, scan → consent → use) and B deferred capsule; one trust model	VII-5.3–5.5
11	Fast-Path consent	Single-action collapse only when all-green; standing scopes never bundled	VII-5.6, VII-10.5.1
12	pBEAP™ relationship model	Persistent P2P relationship until revocation; no conversion to role symmetry, no silent escalation, no execution authority; persistence/symmetry/transport are independent axes	VII-12.1
13	Publisher reachability (practice/hotel case)	Grants = delivery + preparation rights; standing action scopes deliver prepared actions; execution always consent-gated	§10
14	Auto-execution under standing scopes	Rejected — preparation right, never execution right; consent shrinks, never disappears	VII-10.1
15	Grant lifetime default	No expiry until revoke; sessions organize but never govern lifetimes; no automatic teardown	VII-10.7.1, VII-11.1

#	Question	Resolution	Ref
16	Agreed limits	single_use (dissolves on consented execution) and ttl, critical extensions, advanced UI, stricter-declaration-wins	VII-10.8
17	Revocation semantics	Unilateral, immediate, silent toward publisher; severs rights, keeps history; re-handshake reanimates nothing	VII-10.7.2–.4
18	Red WR Codes®	Color = human affordance, sealing = control	VII-4.8
19	Publisher region field	Two-entity attestation (domain_anchor / offering_entity) + delegation; P0/P1/P2; DNS+WebPKI dual channel; enforcement later as P2+ without schema change	§8
20	Geofencing	Local policy engine over attested jurisdictions; importable sanctions sources; min P1 recommended; never applies to private persons	§9
21	Session clustering	Local, keyed on attested publisher identity; hierarchy only on verified session reference; never silent merge	VII-11.2–11.4
22	Payload safety	Schema validation before rendering (MUST); restricted template language registered as later hardening	VII-10.5.4
23	Publishing entitlements	Account classes Free/Pro/Publisher/Enterprise; pBEAP™ and org profiles Publisher+Enterprise only; consumer role class-independent; enforced at registry and authoring surface; Enterprise post-MVP (Silicon Cargo)	§4.5, VII-15.8
24	Ingress evolvability	ingress_path is a registry-backed identifier (optirando.ingress.*), preserved-and-ignored when unknown, never semantic; new triggers are registry entries, never core changes	VII-3.1, VII-4.6
25	Declaration evolvability	Core carries a generic declarations container; the capability declaration is the registered namespace optirando.decl.capability v1; future capability classes are declaration namespaces, never core fields	VII-3.1, VII-5.1
26	Initiation vs. profile	Initiation classes (Annex IX: public_bearer, targeted_bound) are orthogonal to profiles; fast public initiation always establishes pbeap_publisher; hash-pinned consent required for the fast-path collapse	VII-4.15, VII-5.6
27	Verification state model	Attestation levels are append-only local assessments (Annex IX model), never stored properties of handshake/publisher/credential; imprint + badge render current assessment state; P2 arrives as an additional verifier module without migration	VII-8.9, VII-14.7

17. Version History

Version	Date	Change
1.0	2026-07-11	Final draft for Zenodo publication: permanent handshake wire-format core (signed core, signature list 1..n, extensions container with criticality), SSO identity binding, profile registry with four initial profiles and account-class entitlements, capability declaration Variants A/B with Merkle-root commitment and offline root aging, adapter manifests with canonical naming and bridge adapters, verification

Version	Date	Change
1.1	2026-07-13	chain and levels, publisher attestation (two-entity model, delegation, P0/P1/P2, honest semantics), local admission policy (geofencing), the grant model (execution boundary, delivery rights, standing action scopes, vocabularies, lifecycle, agreed limits), sessions and identity clustering, the pBEAP™ invariant and role asymmetry, distribution paths, UI specification (Capability Diff, dual provenance, commit flow, relationship overview, hard-fail rendering), composer authoring requirements, resolved-design-decisions record. Refactor-resistance and initiation-alignment revision; the grant model, profiles, verification chain, and pBEAP™ invariant are unchanged. Core evolvability: <i>ingress_path</i> converted from a closed enum to a registry-backed identifier (<i>optirando.ingress.</i>) with <i>preserve-and-ignore semantics</i>; <i>initial entries</i> extended by <i>relay_code_claim</i> and <i>optirando_code_entry</i>, <i>wr_ad</i> reserved (VII-3.1, VII-4.6, §4.2 table). <i>capability_declaration</i> generalized to a declarations container [{<i>ns</i>, <i>version</i>, <i>payload</i>}] under container discipline, with the Section-5 block carried unchanged as <i>optirando.decl.capability v1</i> (VII-3.1, VII-5.1). New §3.4 reserves extension namespaces: <i>optirando.handshake.prior_ref</i> (escalation lineage, wired into VII-4.7), <i>optirando.credential</i>. (post-establishment Credential Attachments — append-only second signature chain; establishment-time container delimitation stated), <i>optirando.transport.*</i> (VII-3.12). New §4.6: initiation classes (Annex IX) declared orthogonal to profiles; bidirectional/full-context vocabulary bound exclusively to the symmetric profiles (VII-4.15). VII-5.6 extended: fast public initiation requires consent over the hash-pinned first-artifact reference with first-interaction match (Annex IX). New VII-8.9: verification state as separate, append-only assessment records (Annex IX model) — never stored on handshake, publisher, or credential; verifier modules additive. New VII-14.7: relationship imprint rendering (both entities' imprint fields + attestation badge) under the store-reference / render-current-assessment rule. Terminology: "secret path" renamed Private Distribution Path throughout (§0, VII-8.8, VII-12.6, §13.2, VII-15.6, decision #6); VII-4.3 reworded without the retired "hidden path" phrase; retired "Forward Preparation" citations replaced by the WR Preparation stages with corrected references to Annex VI (VII-10.1, VII-11.6, VII-14.4). Terminology section gains the Initiation Class entry. Decisions #24–#27 recorded.

License Notice

This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™